DB Monitoring: стратегия деплоя и отката

Домашнее задание B10 «Стратегии деплоя и релиз-процессы» (слайд 45; практика — слайды 33–38), Issue #14. Сверено с `master` `f891234` на 10.11.2026. Это проектное решение: маршрутизация трафика, автоматическое сравнение версий и испытание отката не реализованы и не проводились; этап «100 % трафика» — целевое состояние схемы.

1. Что выпускается

Приложение — Flask со встроенным планировщиком: сбор метрик источника (micro-batch, по умолчанию раз в 15 мин), оценка аномалий, уведомления, ночное обучение. Docker Compose поднимает PostgreSQL-источник, TimescaleDB-хранилище, Flask и по профилю `inference` отдельный FastAPI-инференс без состояния (реестр моделей только для чтения). Выпускаются два независимых артефакта: **образ приложения** (semver-тег; `APP_VERSION` виден в `/healthz.version`) и **версия модели** Isolation Forest в реестре (`if-<SHA-256>`, схема признаков `1.0.0`). Версия модели включается для каждого источника отдельно — привязкой (`bindings`) в конфигурации адаптера. Основной поток работы — тики сборщика, а не HTTP-запросы пользователей.

2. Выбор: blue-green для приложения + canary по источникам для модели

Приложение — blue-green с последовательным переключением. Ошибка кода проявляется сразу (`/healthz`, 5xx, сбой сборщика), поэтому важнее быстро вернуть рабочую версию, чем постепенно набирать уверенность (вопрос слайда 38). Держать две версии Flask под нагрузкой одновременно нельзя: каждая запускает свой планировщик (двойной сбор и ночное обучение), а обновление хранилища требует остановить старые процессы (migrations.md). Поэтому «зелёная» версия проверяется в отдельном изолированном Compose-проекте на восстановленной копии данных, а рабочий стенд переключается заменой контейнера между тиками; «синий» образ остаётся точкой отката. Короткий простой при замене (оценка — до минуты) допустим для внутреннего инструмента с периодическим сбором.

Модель — shadow, затем canary по источникам. Ошибка модели «тихая» (слайд 12): сервис работает, но хуже находит инциденты. Нужны постепенная проверка на реальных данных и сравнение с предыдущей версией. Основа уже есть: версия задаётся для каждого источника, оценки хранятся с ключом версии, откат — возврат привязки без переобучения. Единица «трафика» — источник (подключение + схема + таблица); один источник обслуживает одна версия.

Отклонено. Canary приложения требует маршрутизатора трафика, которого нет; у внутреннего инструмента мало пользователей, и доля 5–25 % не даёт статистики, а две версии делили бы хранилище и фоновые задания. Blue-green модели «сразу на все источники» не даёт выигрыша: быстрый откат модели уже обеспечивают привязки, а одновременное переключение скрывает ухудшение до пропущенных инцидентов.

3. Схема процесса: от «код готов» до «100 % трафика»

[P] — реализовано в репозитории; [P*] — реализовано, но в этом репозитории не запускалось; [П] — предлагается.

```
КОД ГОТОВ: PR в master
  ▼
1 CI [P]: Ruff; pytest 3.11–3.13; integration со сборкой образа --no-cache; Compose-сценарий
  (technical_pass + check-outcome); после слияния — E2E на master ──────── провал ──► СТОП, исправить
  ▼ успех
2 Тег vX.Y.Z → release.yml [P*]: образ X.Y.Z, latest, previous. Записать digest и номер
  предыдущей версии; разворачивать только неизменяемый X.Y.Z, не latest
  ▼
3 GREEN [П]: свежий бэкап → восстановление в отдельный Compose-проект → новый образ один раз
  (миграция) → инварианты БД, /healthz, версия, smoke, тик сборщика, /v1/model привязок
  ──────────────────────────────────────────────────────────────── провал ──► СТОП, BLUE не тронут
  ▼ успех
4 ПЕРЕКЛЮЧЕНИЕ между тиками [П; команды Compose есть]: бэкап рабочих БД → stop app (blue) →
  APP_IMAGE=<образ>:X.Y.Z docker compose up -d --no-build app (green); blue-образ сохранён
  ▼
5 КОНТРОЛЬ 30 мин (2 тика), затем 24 ч [П]: строки 1–4 таблицы §4
  ├─ порог отката ──► ОТКАТ приложения (§5 А)
  ├─ мало данных ──► ДЕРЖАТЬ, продлить окно
  ▼ норма
  ПРИЛОЖЕНИЕ ПРИНЯТО (100 % трафика с шага 4); привязки моделей при этом не менялись
  ▼ отдельный релиз модели
6 Допуск реестра [P]: test ≠ train; recall ≥ 80 %, FPR ≤ 10 %, задержка ≤ 1 тик — не допущена ──► СТОП
  ▼ допущена
7 SHADOW ≥ 24 ч [П]: новая версия оценивает те же наблюдения без уведомлений; сравнение
  со старой по строкам 5–8 ──────────────────────────────────────────── хуже ──► СТОП, не выпускать
  ▼ не хуже
8 CANARY [P: привязки; П: процедура]: 1 источник (~10 %) ≥ 24 ч → 50 % ≥ 24 ч → 100 %;
  на каждом шаге по строкам 5–8: продолжить / ДЕРЖАТЬ / ОТКАТ привязок (§5 Б)
  ▼
100 % ТРАФИКА: образ X.Y.Z и новая модель у всех источников; прежние образ и версия модели
хранятся не менее 7 дней как точка отката
```

Реализовано и проверялось (CI, тесты, протоколы)	Есть в коде, но здесь не запускалось	Нет — предлагается этим документом
CI `tests.yml`/`lint.yml` и Compose-сценарий #13; реестр версий и допуск `admitted()` #5; привязки и оценки с ключом версии #12; `/healthz`, `/metrics`, алерты `HttpErrorRatioHigh` #15 и `ModelAnomalyRateHigh` #16; backup/restore (тесты)	`release.yml`, `promote.yml` — тегов `v*.*.*` нет; runbook отката — dry-run не зафиксирован; процедура миграций на рабочих данных	Маршрутизатор и две рабочие среды; фоновая shadow-оценка и сравнение версий; допущенная модель (её нет, §6); внешняя доставка алертов

4. Критерии успешности и пороги

Новая версия должна быть не хуже предыдущей: приложение сравнивается с окном той же длины до релиза, модель — с предыдущей версией на тех же наблюдениях. «10 % при ≥ 10 запросах» и «3 из 20» — действующие правила #15 и #16, остальные пороги — **проектные**, до калибровки. Без минимума данных решение не принимается и этап продлевается: «нет данных» ≠ «норма».

№	Показатель и источник	Единица, окно	Продолжить	Держать, разобрать	Откат
1	Доступность: `/healthz`, Docker HEALTHCHECK раз в 30 с	доля успешных проб; 30 мин	100 %, `version` = X.Y.Z	1–2 неуспешные пробы	3 подряд (`unhealthy`) или новый `down` компонента
2	HTTP 5xx: `http_requests_total` без `/metrics`	%; 5 мин, ≥ 10 запросов	≤ 1 % и ≤ доли до релиза + 1 п. п.	> 1 % и ≤ 10 %	> 10 % (`HttpErrorRatioHigh`) или 5xx на smoke
3	Время ответа p95: `http_request_duration_seconds`	с; 15 мин, ≥ 30 запросов	≤ 1,2 × p95 до релиза	> 1,2 ×	> 2 × или > 2 с два окна подряд
4	Сбор метрик: `collector_runs_total{result}`	ошибочные тики; 2 тика	0, если до релиза было 0	1 ошибка	2 тика подряд у источника, исправного до релиза
5	Ошибки инференса: `inference_predictions_total`, `outcome` ≠ `ok`	%; 1 ч, ≥ 20 вызовов	≤ 1 %	> 1 % и ≤ 5 %	> 5 % или `model_unavailable` новой версии
6	Качество и свежесть входов: `inputs.conclusive` (#16)	да / нет	`true`	`false` — этап не продвигается	сбой сбора после релиза — строка 4
7	Доля аномалий против baseline версии: `ModelAnomalyRateHigh`	из 20 последних результатов	< k (demo-версия: k = 3)	≥ k — проверить источник	только вместе со строкой 8
8	Расхождение с предыдущей версией на тех же наблюдениях (shadow, canary)	наблюдения; ≥ 96 на источник (≈ 24 ч)	≤ 2 новых флагов; флаги на подтверждённых инцидентах сохранены	> 2 новых флагов или расхождения не разобраны	пропущен подтверждённый инцидент или firing строки 7 только у новой версии при `conclusive` входах

Обоснование: 1 % — в норме 5xx около 0 (#15); 1,2 × — запас на шум и корзины гистограммы (p95 точна до границы корзины); 96 наблюдений — сутки сбора; «новый флаг» — аномалия у новой версии при норме у старой, 2 из 96 ≈ 2 % — порядок FPR demo-модели на test (1,66 %, #13). Precision, recall и FPR считаются только на размеченном test (этап 6); по неразмеченному потоку доля аномалий и расхождения — косвенные сигналы, а не оценка точности.

5. План отката

Запуск и решение. Откат запускается при любом условии столбца «Откат» или при потере либо порче данных. Приложение по порогу откатывает ответственный за релиз — роль DevOps (M4 MLOps-roadmap) — без дополнительного согласования; модель — ML-инженер; инциденты для строк 7–8 подтверждает инженер данных; продолжение после «держать» решает ответственный за релиз.

А. Приложение (цель ≤ 10 мин, с восстановлением БД ≤ 2 ч — плановые значения, не измерены). Зафиксировать время, `/healthz.version`, метрики окна и журналы `app`. Если причина в функции под флагом (сейчас только `forecast`) — `FF_FORECAST=false` и перезапуск `app` (шаг 3 runbook). Иначе `APP_IMAGE=<образ>:<предыдущая X.Y.Z> docker compose up -d --no-build app` — номер с этапа 2, а не плавающий `previous`. Если релиз изменил схему хранилища, старый код на неё не запускать: предрелизный бэкап восстанавливается в новую БД, затем переключается `MONITOR_DB_URL` (раздел «Откат» migrations.md); данные, записанные после бэкапа, переносятся только отдельным решением.

Б. Модель (цель ≤ 1 тик сбора, 15 мин). Вернуть в `bindings` прежние `model_version` и `feature_schema_version`, сначала у canary-источников. По коду конфигурация читается при каждом обращении, перезапуск не нужен (не испытано). Версии в реестре не удаляются; `activate` адаптер не переключает. Реестр сверяет схему признаков и версию scikit-learn и при несовпадении отказывает без fit (`model_unavailable`), поэтому при откате образа с другой scikit-learn или схемой сначала проверяется `/v1/model` прежней версии.

В. Обработанные запросы и результаты. Запросы во время замены контейнера получают ошибку и повторяются пользователем; пропущенный тик не заполняется. Оценки хранятся по ключу «источник + время + версия модели + схема» (`ON CONFLICT DO NOTHING`): результаты новой версии остаются для разбора и не смешиваются с прежними, дашборд снова читает прежнюю пару, а она досчитывает недостающие точки без повторных уведомлений. Отправленные уведомления новой версии не отзываются — их список вносится в запись об инциденте. Собранные новой версией метрики не удаляются; при дефекте сборщика интервал помечается.

Г. Подтверждение. `/healthz` 200 с прежней `version` 20 проб подряд (10 мин); 15 мин без новых 5xx, алерт `resolved`; успешный тик сборщика у всех источников и `outcome=ok` у всех привязок; запись об инциденте.

6. Ограничения

- `release.yml` не запускался: тегов `v*.*.*` нет. README и runbook называют образ `ghcr.io/aleksandr-novikov/db-monitoring`, workflow публикует `ghcr.io/<владелец>/<репозиторий>`, а Compose берёт образ из `APP_IMAGE`: `docker pull …:previous && docker compose up -d app` образ не сменит. `/healthz?strict=true` из runbook при rate-limit `memory://` (по умолчанию) по коду всегда даёт 503. Правка runbook — отдельная задача.
- Допущенной модели нет (#9: FPR 13,27 %; demo-модель #13: `quality_admitted=false`): этап 6 сейчас заканчивается «СТОП», допущенной точки отката модели нет.
- Откат не испытывался. Legacy IF источников без привязки ночью перезаписывает joblib без версий — откат образа его не вернёт. Ручная оценка неактивной версией новейшей точки подавляет уведомление по ней: shadow через UI — только по обработанным точкам.